

Projects in Practice

From **Unknown Unknowns** to **Known Knowns** through **Known Unknowns**.

Identifying Unknown Unknowns

When we start a project, we're in the "**unknown unknown**" state: we don't even know what we don't know yet.

- We need tools and rules to move from **unknown unknowns** to **known unknowns** as quickly as possible.
- The goal isn't to eliminate every unknown — it's to reduce the important ones early, and plan for the rest.

Iterative Process

We work in two iterations (sprints).

- **Sprint 1** aims to surface unknown unknowns quickly — we change features/requirements when prototypes or the MVP show they're not feasible.
- **Sprint 2** builds out the features based on what we learned.

Prototypes

Build a prototype when similar software is unfamiliar — to test feasibility quickly.

- **Throwaway:** Answer the open question; production-grade architecture, design, and tests are unnecessary.
- **Potential product:** Keep a basic data model and maintain code quality from the start.
- Expect throwaway code to change or be discarded later.

MVPs

An MVP (Minimum Viable Product) is a working product with the minimum useful features.

- Later iterations refactor the design and add features based on it.
- *(In this course, your MVP must include the necessary design, tests, and documentation.)*
- Your first sprint goal is to deliver a working MVP quickly, then improve it in later iterations.

Definition of Done and Dogfooding

- Agree on **Done**: reviewed code, passing tests, satisfied acceptance criteria, updated documentation, and no critical known defects.
- **Dogfooding**: use your own software to find usability problems and missing features.
- **Milestones** are checkpoints; the **deadline** is the final constraint.
- Tests and metrics support quality assessment; counts alone do not prove quality.

Aim for High-Quality

As professional software engineers, we should aim to build high-quality software — not just working software.

- The software that is easy to fix bugs
- The software that is easy to add features

Focus also on the software that's also:

- **Correct, usable, and reliable**
- **Also Secure**

Good architecture and design make solving real problems sustainable over time.

Professor (as Manager)

- **Manage People & Politics:** Navigate complexity and team dynamics.
- **Evaluate Teams:** Assess tech leads and team members.
- **Support Problem-Solving:** Provide guidance and resources to resolve issues.

Team Leaders (Tech Leads)

- **Manage Team Complexity:** Solve problems by organizing people and tasks.
- **Report Upward:** Keep managers informed of team progress.
- **Set Direction:** Define goals and schedules.
- **Drive Progress:** Ensure team members move forward.
- **Manage Artifacts:** Code, tests, and documents.

- **Lead Weekly Meetings:** Present the team's weekly progress — or delegate to a team member if you can't attend.
- **Lead Presentations:** Guide the team through PPP, S1P/S1R/S2R, and FP.

The acronyms introduced here are explained in the following slides.

Team Members

- **Tackle Complexity:** Solve problems through design and implementation.
- **Model the Data:** Define the data and how it flows through modules.
- **Design Interfaces:** Design the modules and interfaces that use that data.
- **Build & Test:** Write the code and the tests that verify it.

- **Raise Issues:** Signal problems early to get them resolved.
- **Mentor Others:** Support newer teammates.

This course expects professional ownership: work autonomously, explain your decisions, and raise issues early.

No Surprises

Unexpected problems break trust more than difficult problems.

1. **Communicate Early:** Raise issues as soon as they appear.
2. **Be Transparent:** Share progress, blockers, and risks openly.
3. **Avoid Last-Minute Shocks:** No one should be blindsided.

Stay Constructive Under Pressure

Stay respectful and fact-based, especially under pressure.

1. **Manage Emotions:** Respond with facts, not frustration.
2. **Focus on Solutions:** Discuss data and solutions, not blame.
3. **Build Trust:** Be steady and speak up about problems.

Be a Professional

Professionalism is about actions and results, not titles.

1. **Deliver on Promises:** Do what you say, on time.
2. **Respect Others:** Treat everyone with fairness and courtesy.
3. **Take Responsibility:** Own mistakes and learn from them.
4. **Strive for Quality:** Aim for excellence in every task.

Understand Others before Being Understood

Listening deeply builds trust faster than speaking first.

1. **Seek Context:** Learn the origin of the problem and who's involved.
2. **Listen Actively:** Let others feel heard, and acknowledge their concerns, before offering input.
3. **Respond Thoughtfully:** Solutions land better when people feel understood.

Start Early and Reduce Uncertainty

We can plan well only once we've reduced the important unknowns.

1. **Start Early:** Begin quickly to reveal what you don't know.
2. **Prototype First:** Test feasibility before the MVP, to make sure you're solving the right problem.
3. **Seek Clarity:** Work hard to gain a clear vision of the problem as you go.

Deliver a Small, Working Result First

A rough solution that works is more valuable than a perfect one that never ships.

1. **Start Simple:** Build a small vertical slice that delivers real end-to-end value, not isolated layers.
2. **Prove Value:** Show it solves the right problem before refining it.
3. **Refine Later:** Once effectiveness is clear, optimize for speed and polish.

Improve Using Evidence and Feedback

Good systems and habits make future work easier, not harder.

1. **Iterate:** Use feedback and results to improve the design and the process.
2. **Reduce Friction:** Build habits and tools that lower mental load, not add to it.
3. **Stay Consistent:** Aim for reliable results regardless of who's doing the work.